EXPERIMENT : 1

1.a Analyze the trend of data science job postings over the last

AIM:

Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.

DESCRIPTION:

Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {'Year': list(range(2010, 2021)),

'Job Postings': [150, 250, 370, 520, 700, 1100, 1500, 2200, 2500, 3400, 4700]}


df = pd.DataFrame(data)

plt.plot(df['Year'], df['Job Postings'], marker='o')

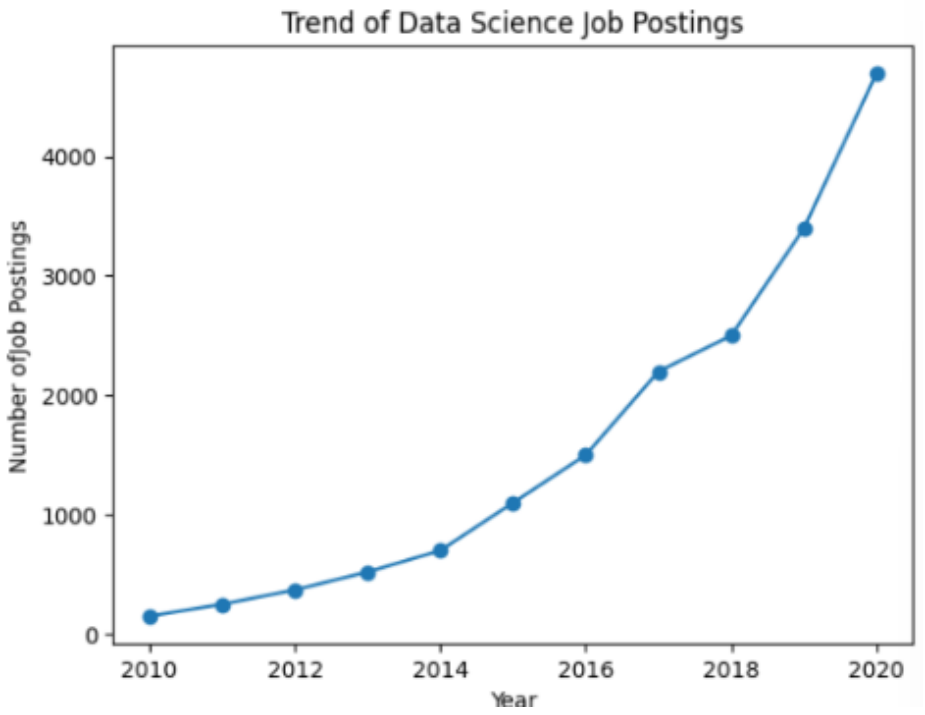
plt.title('Trend of Data Science Job Postings')

plt.xlabel('Year')

plt.ylabel('Number ofJob Postings')

plt.show()
```

OUTPUT:



RESULT:

The graph shows that data science job postings increased from 2015 to 2025. The highest number of job postings was in 2025.

1.b Analyse and visualize the distribution of various data science roles

AIM:

Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

DESCRIPTION:

Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

CODE:

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer','Business Analyst']

counts = [200, 600, 350, 400, 150]

plt.bar(roles, counts)

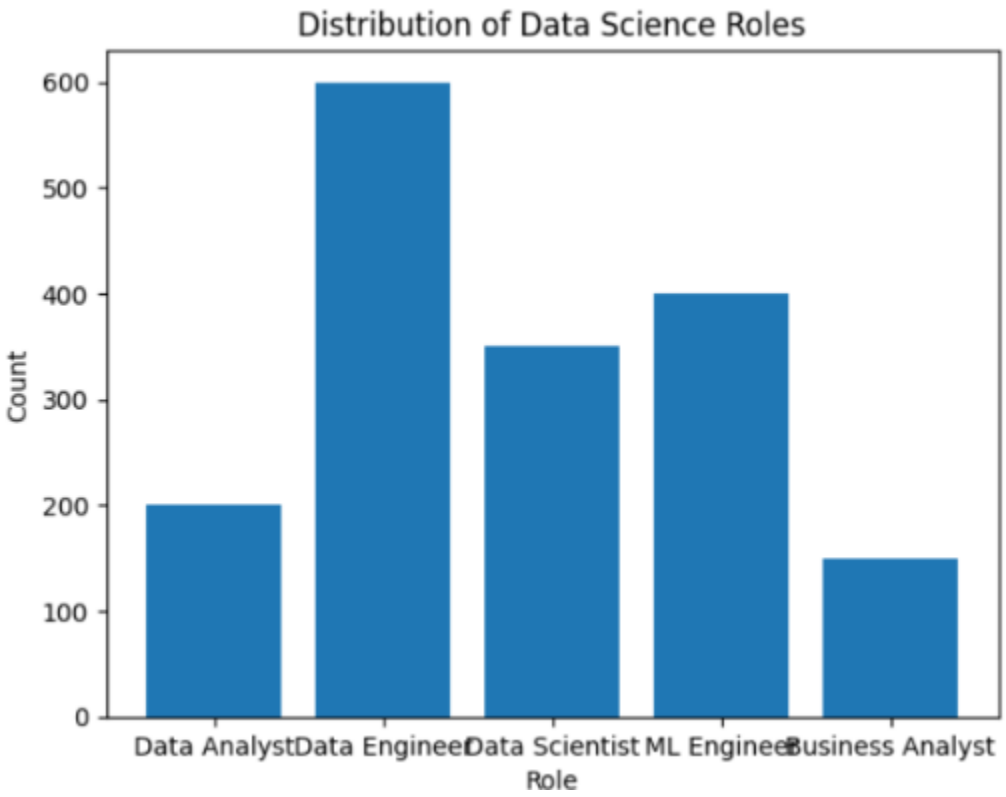
plt.title('Distribution of Data Science Roles')

plt.xlabel('Role')

plt.ylabel('Count')

plt.show()
```

OUTPUT:



RESULT:

Data scientist is the occurring rate in the dataset(count=3).Data analyst(3),Data Engineer(2) and ML engineer(1) are equally dataset.

1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data.

AIM:

Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

DESCRIPTION:

Create small datasets for each type and explain their characteristics.

CODE:

```
# Structured data
structured_data = pd.DataFrame({
'ID': [101, 102, 103],
'Name': ['Reena', 'Rocky', 'Ravi'],
'Age': [20, 25, 35]
})
print("Structured Data:\n", structured_data)

# Unstructured data
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image or a video file"
print("\nUnstructured Data:\n", unstructured_data)

# Semi-structured data
semi_structured_data = {'ID': 101, 'Name': 'Reena', 'Attributes': {'Height': 153, 'Weight': 56}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

OUTPUT:

Structured Data:

	ID	Name	Age
0	101	Reena	20
1	102	Rocky	25
2	103	Ravi	35

Unstructured Data:
This is an example of unstructured data. It can be a piece of text, an image or a video file

Semi-structured Data:
{'ID': 101, 'Name': 'Reena', 'Attributes': {'Height': 153, 'Weight': 56}}

RESULT:

The experiment successfully differentiated structured, semi-structured, and unstructured data based on their format and organization. Structured data has a fixed tabular format, semi-structured data uses flexible tags/keys, while unstructured data has no predefined format.

1.d Conduct an experiment to encrypt and decrypt given sensitive data.

AIM:

Conduct an experiment to encrypt and decrypt given sensitive data. Use the cryptography library fernet.

DESCRIPTION:

Use the cryptography library to encrypt and decrypt a piece of data.

CODE:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Bitty got some Butter")
token
b'...'
f.decrypt(token)
b'Bitty got some Butter'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Bitty got some Butter"
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

OUTPUT:

Original Data: b'Bitty got some Butter'
Encrypted Data:
b'gAAAAABqe2CsbTafe1qhRy7k2CpSPN-gIXTzKNAyibCrsP4YVMVyuw3sk_H9zWC6e
tY7Qckxh7xC59Pcg3sm9tpiLtU8bwyBLt3qTPKET4GJT9cv_fN3EOI='
Decrypted Data: b'Bitty got some Butter'

RESULT:

The fernet symmetric key algorithm successfully encrypted back original data text,verifying that any turning with the cipherset would cause description to fail.